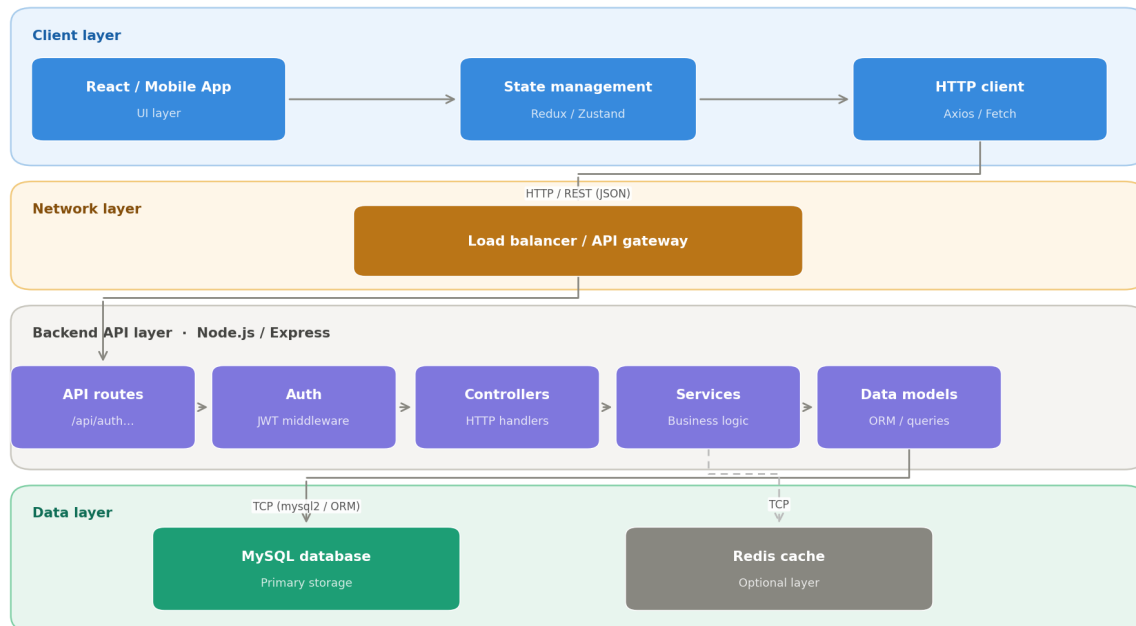


System Architecture Design

1. Architecture Diagram



2. API Connection with Frontend

The frontend communicates with the backend via **RESTful HTTP requests**.

- **Data format:** JSON is used for payloads and responses.
- **Authentication:** The backend issues a JWT upon successful login. The frontend stores this token (e.g., in `localStorage` or secure `HttpOnly` cookies) and includes it in the `Authorization: Bearer <token>` header of subsequent requests.
- **Client implementation:** The frontend uses an HTTP client (Axios or the native Fetch API) wrapped in service functions to handle requests, responses, and global error handling such as token expiration.

3. Backend Structure

The backend follows a standard **MVC / Layered Architecture** pattern:

- **Routes (/src/routes):** Defines the API endpoints (e.g., /api/auth, /api/groups) and maps them to specific controller functions.
- **Middlewares (/src/middleware):** Intercepts incoming requests for cross-cutting concerns like authentication (JWT verification), input validation, rate limiting, and error handling.
- **Controllers (/src/controllers):** Handles the incoming HTTP request, extracts parameters/body, calls the appropriate service or model, and returns the HTTP response.
- **Models (/src/models):** Represents the data structures and contains direct database queries or ORM logic to interact with the MySQL database.
- **Config (/src/config):** Manages environment variables, database connection pools, and third-party integrations.